МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-13

Выполнил:

А.М. Литвинчук

студент группы ПО-13

Проверил:

А. Д. Кулик

13.02.2026

Брест 2026

Лабораторная работа 5

Цель работы: приобрести практические навыки разработки API и баз данных
Требования к оформлению отчета
Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями):
1) Изложение цели работы.
2) Задание по лабораторной работе с описанием своего варианта.
3) Спецификации ввода-вывода программы.
4) Текст программы (кратко).
5) Выводы по проделанной работе.
Общее задание
1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Задание - 12) База данных Европейские футбольные чемпионаты

src/models/country.py
python
```
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from database import Base


class Country(Base):
    __tablename__ = "countries"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False, unique=True)
    code = Column(String(3), nullable=False, unique=True)

    leagues = relationship("League", back_populates="country")
    teams = relationship("Team", back_populates="country")
    players = relationship("Player", back_populates="country")
```
src/models/league.py
python
```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship
from database import Base
```

```
class League(Base):
    __tablename__ = "leagues"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    country_id = Column(Integer, ForeignKey("countries.id"), nullable=False)

    country = relationship("Country", back_populates="leagues")
    seasons = relationship("Season", back_populates="league")
```

src/models/match.py

python

```
from sqlalchemy import Column, Integer, DateTime, ForeignKey, CheckConstraint
from sqlalchemy.orm import relationship
from database import Base


class Match(Base):
    __tablename__ = "matches"

    id = Column(Integer, primary_key=True)
    season_id = Column(Integer, ForeignKey("seasons.id"), nullable=False)
    home_team_id = Column(Integer, ForeignKey("teams.id"), nullable=False)
    away_team_id = Column(Integer, ForeignKey("teams.id"), nullable=False)
    match_date = Column(DateTime, nullable=False)
    home_goals = Column(Integer, nullable=False, default=0)
    away_goals = Column(Integer, nullable=False, default=0)

    __table_args__ = (
        CheckConstraint("home_team_id <> away_team_id", name="chk_diff_teams"),
    )

    season = relationship("Season", back_populates="matches")
    home_team = relationship("Team", foreign_keys=[home_team_id],
back_populates="home_matches")
    away_team = relationship("Team", foreign_keys=[away_team_id],
back_populates="away_matches")
    player_stats = relationship("PlayerStat", back_populates="match")
```

src/models/player_stat.py

python

```
from sqlalchemy import Column, Integer, ForeignKey
from sqlalchemy.orm import relationship
from database import Base


class PlayerStat(Base):
    __tablename__ = "player_stats"

    id = Column(Integer, primary_key=True)
    match_id = Column(Integer, ForeignKey("matches.id"), nullable=False)
    player_id = Column(Integer, ForeignKey("players.id"), nullable=False)
    team_id = Column(Integer, ForeignKey("teams.id"), nullable=False)

    minutes_played = Column(Integer, nullable=False, default=0)
    goals = Column(Integer, nullable=False, default=0)
    assists = Column(Integer, nullable=False, default=0)
    yellow_cards = Column(Integer, nullable=False, default=0)
    red_cards = Column(Integer, nullable=False, default=0)

    match = relationship("Match", back_populates="player_stats")
    player = relationship("Player", back_populates="stats")
    team = relationship("Team", back_populates="player_stats")
```

src/models/player.py

python

```
from sqlalchemy import Column, Integer, String, Date, ForeignKey
from sqlalchemy.orm import relationship
```

```
from database import Base


class Player(Base):
    __tablename__ = "players"

    id = Column(Integer, primary_key=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)
    birth_date = Column(Date)
    country_id = Column(Integer, ForeignKey("countries.id"))

    country = relationship("Country", back_populates="players")
    stats = relationship("PlayerStat", back_populates="player")
```

src/models/season.py

python

```
from sqlalchemy import Column, Integer, String, Date, ForeignKey
from sqlalchemy.orm import relationship
from database import Base


class Season(Base):
    __tablename__ = "seasons"

    id = Column(Integer, primary_key=True)
    league_id = Column(Integer, ForeignKey("leagues.id"), nullable=False)
    name = Column(String(20), nullable=False)
    start_date = Column(Date, nullable=False)
    end_date = Column(Date, nullable=False)

    league = relationship("League", back_populates="seasons")
    matches = relationship("Match", back_populates="season")
```

src/models/team.py

python

```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship
from database import Base


class Team(Base):
    __tablename__ = "teams"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    country_id = Column(Integer, ForeignKey("countries.id"))
    founded_year = Column(Integer)

    country = relationship("Country", back_populates="teams")

    home_matches = relationship(
        "Match",
        back_populates="home_team",
        foreign_keys="Match.home_team_id"
    )

    away_matches = relationship(
        "Match",
        back_populates="away_team",
        foreign_keys="Match.away_team_id"
    )

    player_stats = relationship("PlayerStat", back_populates="team")
```

src/routers/country.py

python

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from database import SessionLocal
```

```
from crud.country import get_all, get, create, update, delete
from pydantic import BaseModel
from typing import List

router = APIRouter(prefix="/countries", tags=["Countries"])


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


class CountryCreate(BaseModel):
    name: str
    code: str


class CountryRead(BaseModel):
    id: int
    name: str
    code: str

    model_config = {
        "from_attributes": True
    }


@router.get("/", response_model=List[CountryRead])
def list_countries(db: Session = Depends(get_db)):
    return get_all(db)


@router.get("/{country_id}", response_model=CountryRead)
def read_country(country_id: int, db: Session = Depends(get_db)):
    obj = get(db, country_id)
    if not obj:
        raise HTTPException(404, "Country not found")
    return obj


@router.post("/", response_model=CountryRead, status_code=201)
def create_country(data: CountryCreate, db: Session = Depends(get_db)):
    return create(db, data.name, data.code)


@router.put("/{country_id}", response_model=CountryRead)
def update_country(country_id: int, data: CountryCreate, db: Session =
Depends(get_db)):
    obj = update(db, country_id, data.name, data.code)
    if not obj:
        raise HTTPException(404, "Country not found")
    return obj


@router.delete("/{country_id}", status_code=204)
def delete_country(country_id: int, db: Session = Depends(get_db)):
    ok = delete(db, country_id)
    if not ok:
        raise HTTPException(404, "Country not found")
    return
```

src/database.py

python

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
```

```
DATABASE_URL = "postgresql+psycopg2://postgres:Azino888@localhost:5432/spp"

engine = create_engine(
    DATABASE_URL,
    echo=True,
    future=True
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False,
    future=True
)

Base = declarative_base()
```

src/main.py

python

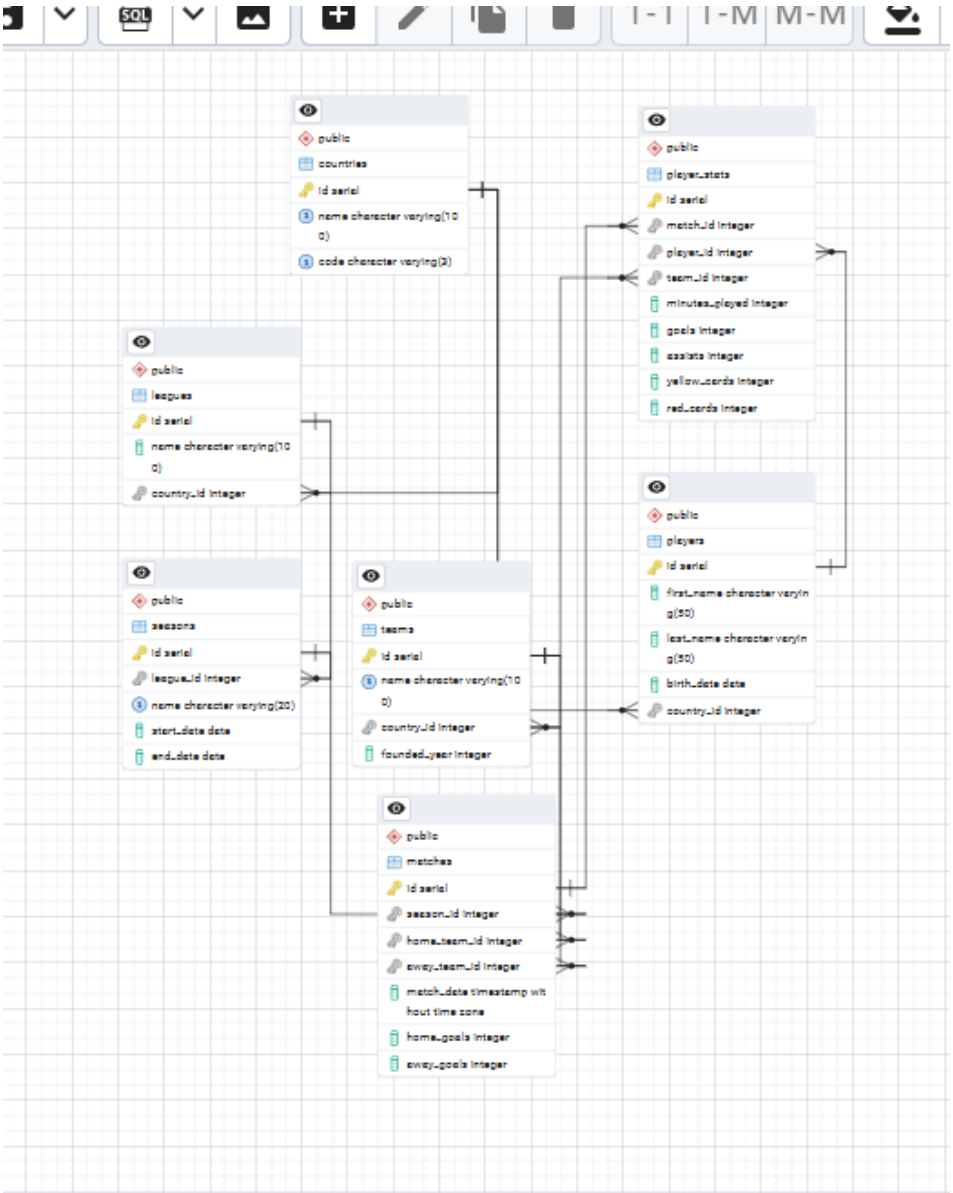
```
from fastapi import FastAPI
from routers.country import router as country_router

app = FastAPI(title="European Football API")

app.include_router(country_router)
```

src/test_connections.py

python

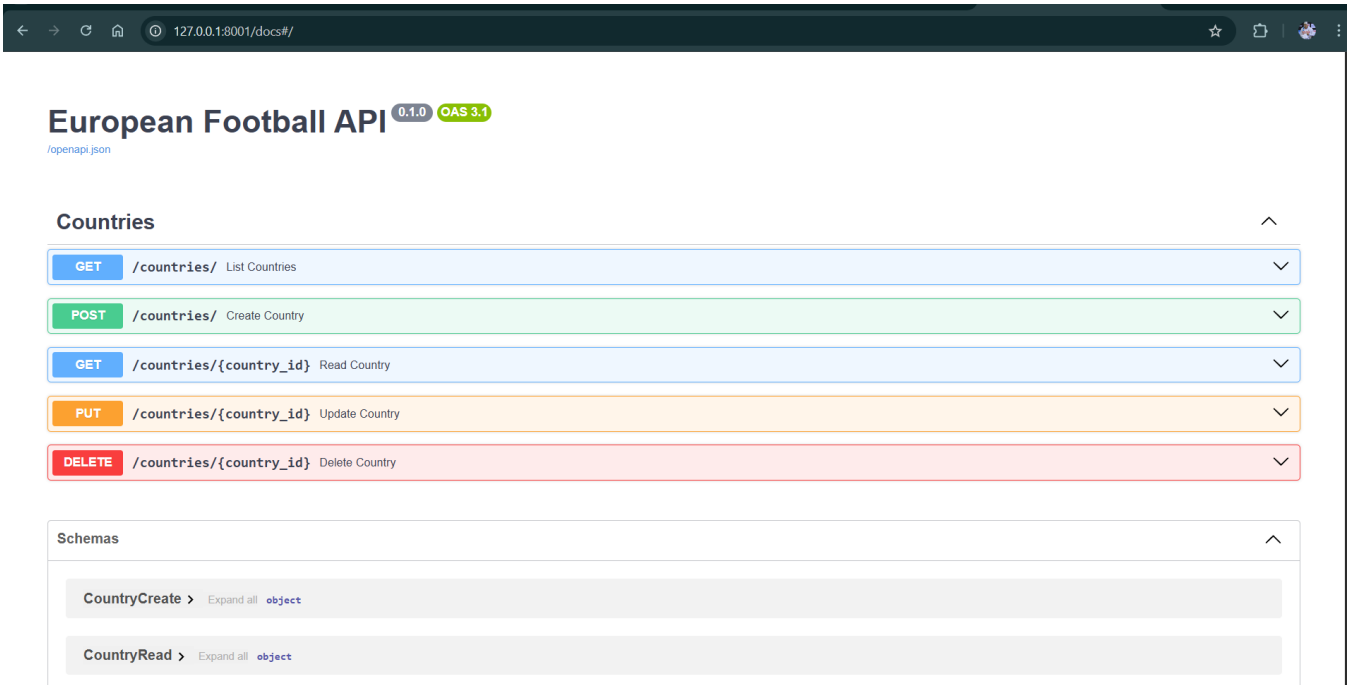
```
from database import engine


def test_connection():
    try:
        with engine.connect() as conn:
            result = conn.execute("SELECT 1;")
            print("Connection OK:", result.scalar())
    except Exception as e:
        print("Connection error:", e)


if __name__ == "__main__":
    test_connection()
```

код posgres



Tables (7)

- countries
- leagues
- matches
- player_stats
- players
- seasons
- teams



Вывод

Разработана и успешно запущена серверная часть API для системы европейского футбола. Настроена структура проекта, реализованы модели SQLAlchemy и CRUD-операции для сущности Country, а также подключена база данных PostgreSQL. Сервер FastAPI корректно запускается, Swagger-документация доступна, что подтверждает работоспособность маршрутов и корректность конфигурации проекта.